

Deployment of 3-Tier Application Using AWS Services

Submitted to - Vikul Sir

Date of Submission – 27/MAY/2026

Submitted by – Rahul Nalathathi

Project Overview

This project demonstrates the manual deployment of a traditional 3-tier web application architecture on AWS.

The application consists of:

1. Frontend Layer (Presentation Tier)
2. Backend Layer (Application Tier)
3. Database Layer (Database Tier)

GitHub Repository:

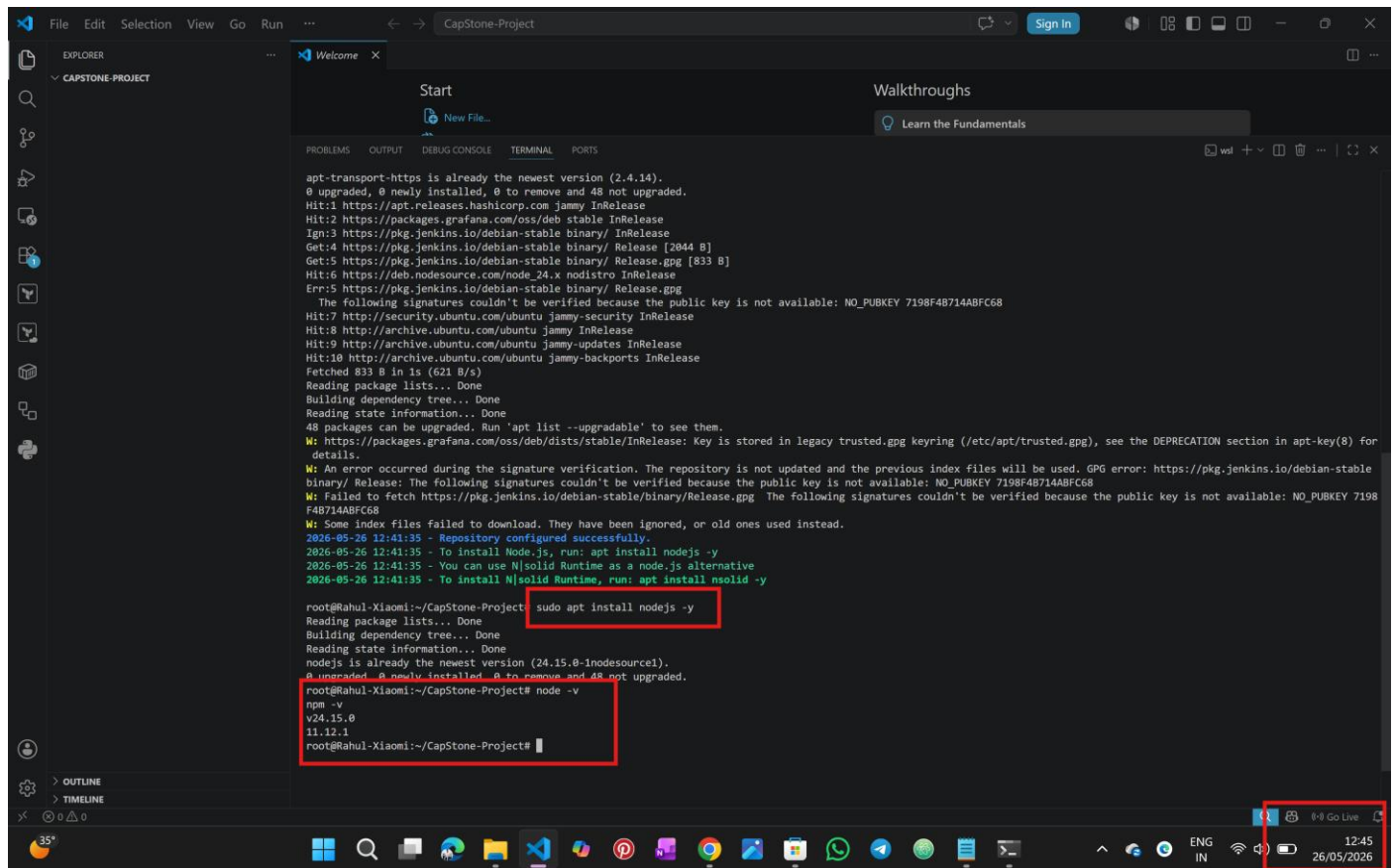
<https://github.com/rahulnalathathi-21/AWS-3-Tier-Architecture/tree/main>

Local Environment Setup

Step 1: Install Node.js(Add NodeSource Repository For Node 24) & Install Node.js 24

```
curl -fsSL https://deb.nodesource.com/setup_24.x | sudo -E bash -
```

```
sudo apt install nodejs -y
```



```
apt-transport-https is already the newest version (2.4.14).
0 upgraded, 0 newly installed, 0 to remove and 48 not upgraded.
Hit:1 https://apt.releases.hashicorp.com jammy InRelease
Hit:2 https://packages.grafana.com/oss/deb stable InRelease
Ign:3 https://pkg.jenkins.io/debian-stable binary/ InRelease
Get:4 https://pkg.jenkins.io/debian-stable binary/ Release [2044 B]
Get:5 https://pkg.jenkins.io/debian-stable binary/ Release.gpg [833 B]
Hit:6 https://deb.nodesource.com/node_24.x nodistro InRelease
Err:7 https://pkg.jenkins.io/debian-stable binary/ Release.gpg
The following signatures couldn't be verified because the public key is not available: NO_PUBKEY 7198F4B714ABFC68
Hit:8 http://security.ubuntu.com/ubuntu jammy-security InRelease
Hit:9 http://archive.ubuntu.com/ubuntu jammy InRelease
Hit:10 http://archive.ubuntu.com/ubuntu jammy-updates InRelease
Hit:10 http://archive.ubuntu.com/ubuntu jammy-backports InRelease
Fetched 833 B in 1s (621 B/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
48 packages can be upgraded. Run 'apt list --upgradable' to see them.
M: https://packages.grafana.com/oss/deb/dists/stable/InRelease: Key is stored in legacy trusted.gpg keyring (/etc/apt/trusted.gpg), see the DEPRECATION section in apt-key(8) for details.
W: An error occurred during the signature verification. The repository is not updated and the previous index files will be used. GPG error: https://pkg.jenkins.io/debian-stable binary/ Release: The following signatures couldn't be verified because the public key is not available: NO_PUBKEY 7198F4B714ABFC68
W: Failed to fetch https://pkg.jenkins.io/debian-stable/binary/Release.gpg The following signatures couldn't be verified because the public key is not available: NO_PUBKEY 7198F4B714ABFC68
W: Some index files failed to download. They have been ignored, or old ones used instead.
2026-05-26 12:41:35 - Repository configured successfully.
2026-05-26 12:41:35 - To install Node.js, run: apt install nodejs -y
2026-05-26 12:41:35 - You can use N|solid Runtime as a node.js alternative
2026-05-26 12:41:35 - To install N|solid Runtime, run: apt install nsolid -y

root@Rahul-Xiaomi:~/CapStone-Project# sudo apt install nodejs -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
nodejs is already the newest version (24.15.0-nodesource1).
0 to upgrade, 0 newly installed, 0 to remove and 48 not upgraded.

root@Rahul-Xiaomi:~/CapStone-Project# node -v
v24.15.0
11.12.1
root@Rahul-Xiaomi:~/CapStone-Project#
```

Step 2: Clone Repository

Git clone [git@github.com:rahulnalathati-21/AWS-3-Tier-Architecture.git](https://github.com/rahulnalathati-21/AWS-3-Tier-Architecture.git)

Backend Setup (Local)

Navigate to Backend Folder

```
cd AWS-3-Tier-Architecture/application-code/app-tier
```

Install Dependencies

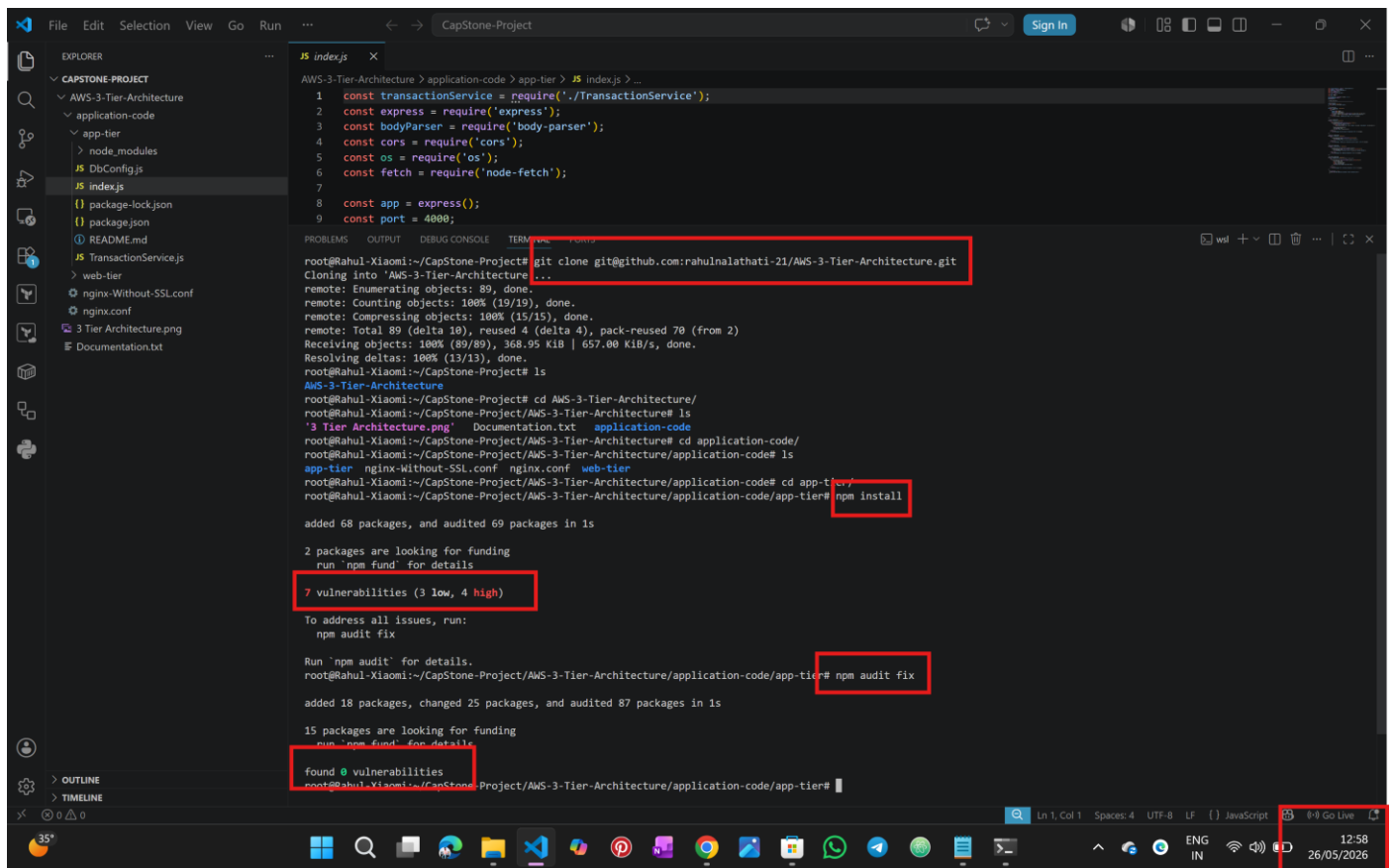
npm install

we have found there are some vulnerabilities, so we have fixed them to zero by using

npm audit fix

Run Backend

node index.js



```
root@Rahul-Xiaomi:~/CapStone-Project# git clone git@github.com:rahulnalathati-21/AWS-3-Tier-Architecture.git
Cloning into 'AWS-3-Tier-Architecture'...
remote: Enumerating objects: 89, done.
remote: Counting objects: 100% (19/19), done.
remote: Compressing objects: 100% (15/15), done.
remote: Total 89 (delta 10), reused 4 (delta 4), pack-reused 70 (from 2)
Receiving objects: 100% (89/89), 368.95 KiB | 657.00 KiB/s, done.
Resolving deltas: 100% (13/13), done.
root@Rahul-Xiaomi:~/CapStone-Project# ls
AWS-3-Tier-Architecture  Documentation.txt  application-code
root@Rahul-Xiaomi:~/CapStone-Project# cd AWS-3-Tier-Architecture/
root@Rahul-Xiaomi:~/CapStone-Project/AWS-3-Tier-Architecture# ls
app-tier  nginx-Without-SSL.conf  nginx.conf  web-tier
root@Rahul-Xiaomi:~/CapStone-Project/AWS-3-Tier-Architecture# cd application-code/
root@Rahul-Xiaomi:~/CapStone-Project/AWS-3-Tier-Architecture/application-code# ls
app-tier  nginx-Without-SSL.conf  nginx.conf  web-tier
root@Rahul-Xiaomi:~/CapStone-Project/AWS-3-Tier-Architecture/application-code# cd app-tier/
root@Rahul-Xiaomi:~/CapStone-Project/AWS-3-Tier-Architecture/application-code/app-tier# npm install

added 68 packages, and audited 69 packages in 1s

2 packages are looking for funding
  run `npm fund` for details

7 vulnerabilities (3 low, 4 high)

To address all issues, run:
  npm audit fix

Run `npm audit` for details.
root@Rahul-Xiaomi:~/CapStone-Project/AWS-3-Tier-Architecture/application-code/app-tier# npm audit fix

added 18 packages, changed 25 packages, and audited 87 packages in 1s

15 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
root@Rahul-Xiaomi:~/CapStone-Project/AWS-3-Tier-Architecture/application-code/app-tier#
```

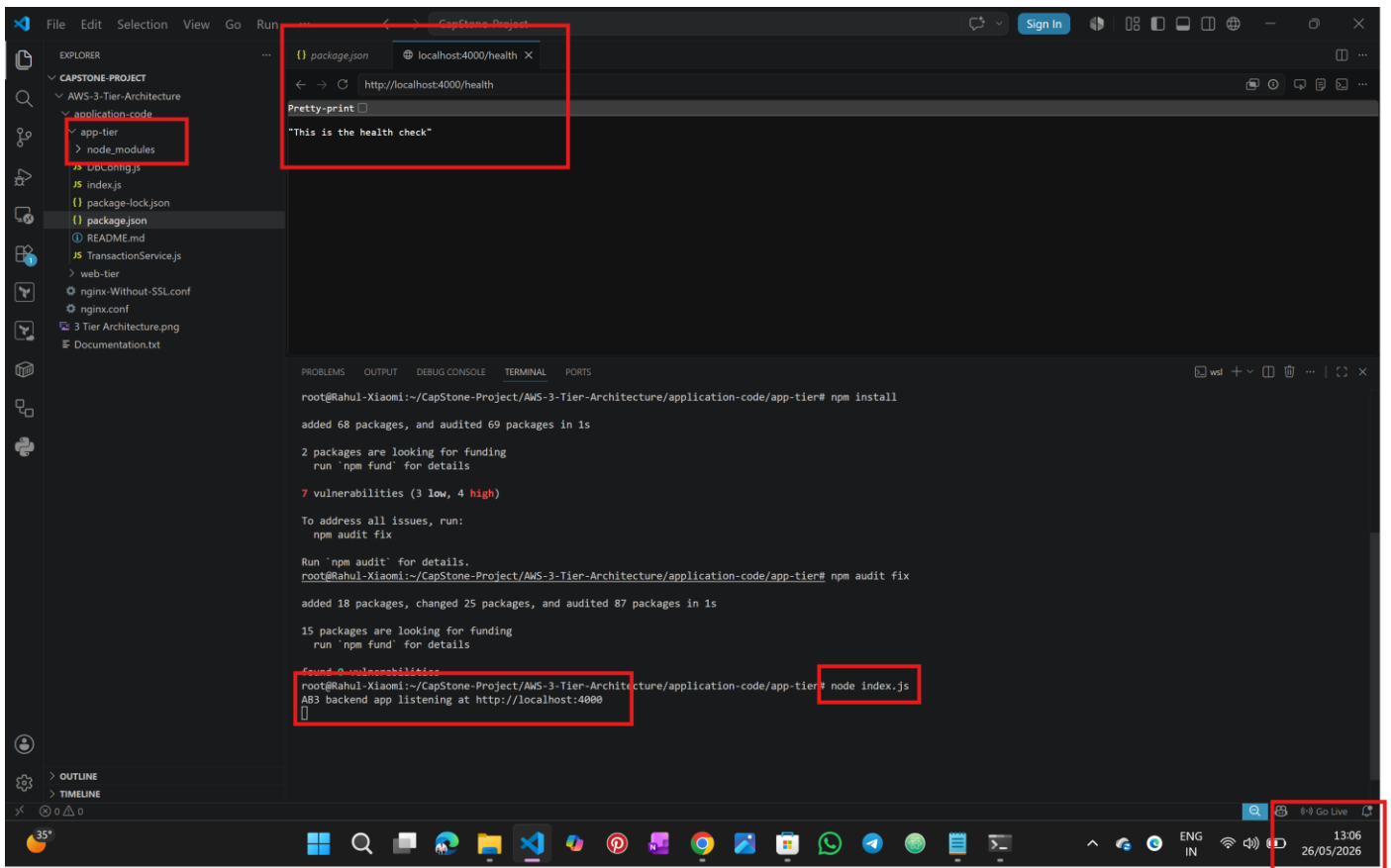
Backend Health Check

Open:

<http://localhost:4000/health>

Expected Output:

"This is the health check"



-----Local setup is working fine now I will do entire project use AWS services-----

AWS Infrastructure Deployment

Login to AWS Console

Step 1: Create VPC

Create VPC workflow

Success

Details

- ✓ Create VPC: vpc-065d841817cb8183e
- ✓ Enable DNS hostnames
- ✓ Enable DNS resolution
- ✓ Verifying VPC creation: vpc-065d841817cb8183e
- ✓ Create S3 endpoint: vpce-0dee844ca23050c9f
- ✓ Create subnet: subnet-01dbd1e09fd93124
- ✓ Create subnet: subnet-0c83683e52433b6fa
- ✓ Create subnet: subnet-03b7fd85c60aea48f
- ✓ Create subnet: subnet-013827fa2f27b943
- ✓ Create internet gateway: igw-06ca692a4baf4881
- ✓ Attach internet gateway to the VPC
- ✓ Create route table: rtb-0b5324034de764fd8
- ✓ Create route
- ✓ Associate route table
- ✓ Associate route table
- ✓ Create route table: rtb-0dd694740830e7836
- ✓ Associate route table
- ✓ Create route table: rtb-0905c3db090a08f73
- ✓ Associate route table
- ✓ Verifying route table creation
- ✓ Associate S3 endpoint with private subnet route tables: vpce-0dee844ca23050c9f

Related VPC resources and services

Discover additional resources you can launch within your VPC to build your network architecture. From security components to connectivity services, explore the building blocks available for your infrastructure.

Client VPN
Networking
AWS Client VPN is a managed client-based VPN service that enables you to securely access AWS resources and...

Resources
Let us know what you would like to create in your VPC
e.g., Instances, Network ACL, Transit gateway...

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms **Cookie preferences**

13:18
26/05/2026

Step 2: Create Security Groups

Backend-sg

Security group (sg-024e3e4dbe2d88d35 | backend-sg) was created successfully

Details

sg-024e3e4dbe2d88d35 - backend-sg

Details

Security group name: backend-sg
Security group ID: sg-024e3e4dbe2d88d35
Description: Backend Security Group
VPC ID: vpc-065d841817cb8183e

Owner: 810648236072
Inbound rules count: 2 Permission entries
Outbound rules count: 1 Permission entry

Inbound rules (2)

Name	Security group rule ID	IP version	Type	Protocol	Port range	Source	Description
-	sgr-0b822ef1d0643dcf2	IPv4	SSH	TCP	22	110.235.225.65/32	-
-	sgr-0f622190eb294f725	IPv4	Custom TCP	TCP	4000	0.0.0.0/0	-

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms **Cookie preferences**

13:24
26/05/2026

Database-SG

us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#SecurityGroup:groupId=sg-0f0a889f37fbfffc1

Security group (sg-0f0a889f37fbfffc1) database-sg was created successfully

sg-0f0a889f37fbfffc1 - database-sg

Details

Security group name database-sg	Security group ID sg-0f0a889f37fbfffc1	Description Security group for RDS MySQL
Owner 810648236072	Inbound rules count 1 Permission entry	Outbound rules count 1 Permission entry

VPC ID
vpc-065d841817cb8183e

Inbound rules (1)

Name	Security group rule ID	IP version	Type	Protocol	Port range	Source	Description
-	sg-0168c06d47be6f61f	-	MySQL/Aurora	TCP	3306	sg-024c3e4d4be2d88d3...	-

13:29 26/05/2026

Step 3: Creating a backend server

us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#InstanceDetails:instanceId=i-05c9f7285a18d908d

Instance summary for i-05c9f7285a18d908d (backend-server)

Updated less than a minute ago

Instance ID i-05c9f7285a18d908d	Public IPv4 address 3.80.148.145 open address	Private IPv4 addresses 10.0.30.78
IPv6 address -	Instance state Running	Public DNS ec2-3-80-148-145.compute-1.amazonaws.com open address
Hostname type IP name: ip-10-0-30-78.ec2.internal	Private IP DNS name (IPv4 only) ip-10-0-30-78.ec2.internal	Elastic IP addresses -
Answer private resource name -	Instance type t3.micro	AWS Compute Optimizer finding Opt-in to AWS Compute Optimizer for recommendations. Learn more
Auto-assigned IP address 3.80.148.145 [Public IP]	VPC ID vpc-065d841817cb8183e (rahu-3tier-vpc-vpc)	Auto Scaling Group name -
IAM role -	Subnet ID subnet-0c83683e52433b6fa (rahu-3tier-vpc-subnet-public2-us-east-1b)	Managed false
IMDSv2 Required	Instance ARN arn:aws:ec2:us-east-1:810648236072:instance/i-05c9f7285a18d908d	
Operator -		

Details Status and alarms Monitoring Security Networking Storage Tags

Instance details

AMI ID ami-0236922087fa98b6e	Monitoring disabled	Platform details Linux/UNIX
AMI name al2023-ami-2023.11.20260514.0-kernel-6.1-x86_64	Allowed image -	Termination protection Disabled
Stop protection Disabled	Launch time Tue May 26 2026 14:06:49 GMT+0530 (India Standard Time) (2 minutes)	AMI location amazon/al2023-ami-2023.11.20260514.0-kernel-6.1-x86_64
Instance reboot migration Default (On)	Instance auto-recovery Default	Lifecycle normal
Stop-hibernate behavior Disabled	AMI Launch index 0	Key pair assigned at launch backend-key
State transition reason -	Credit specification unlimited	Kernel ID -

14:08 26/05/2026

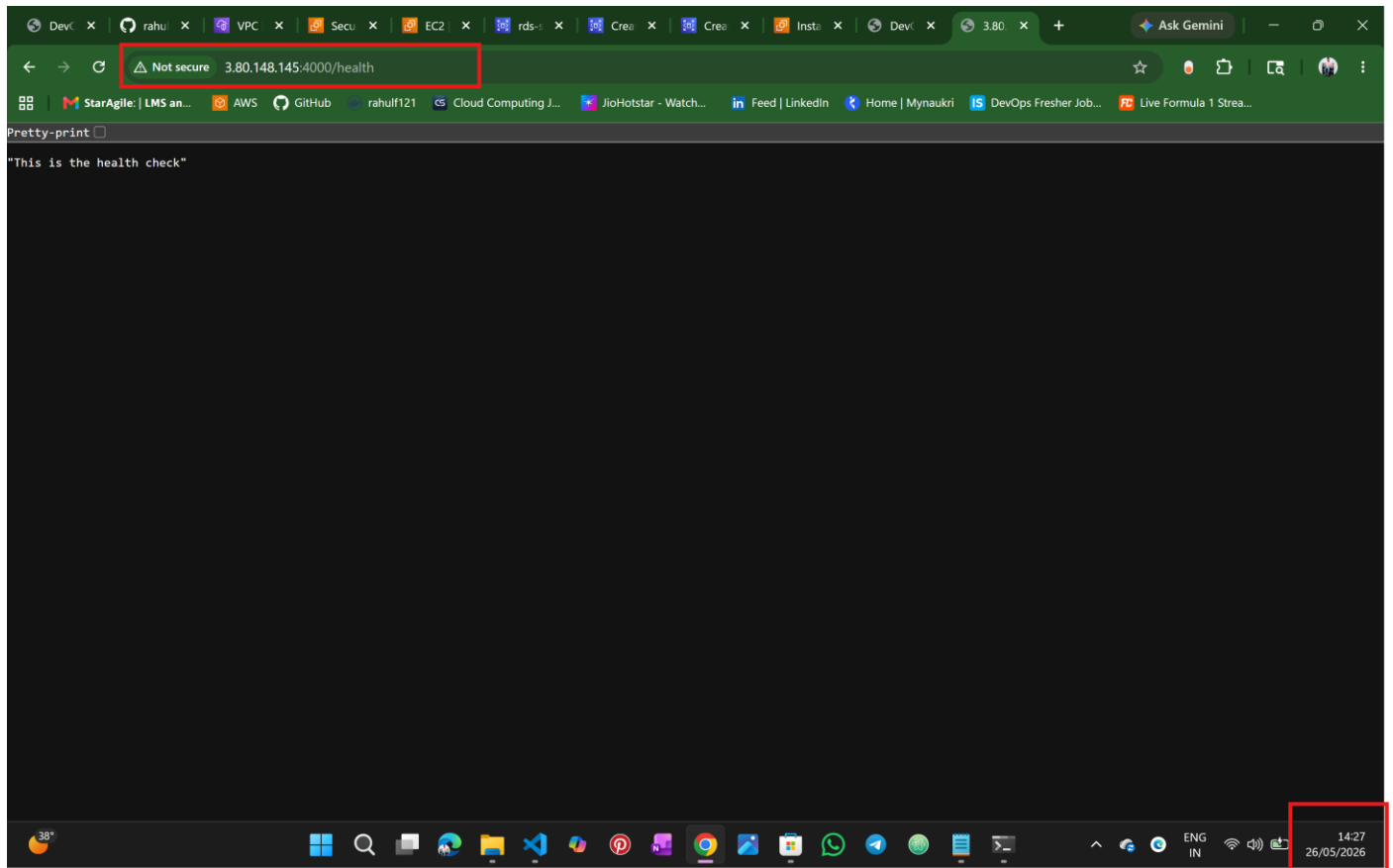
Connecting to the server


```
Complete!  
[ec2-user@ip-10-0-30-78 ~]$ git --version  
git version 2.50.1  
[ec2-user@ip-10-0-30-78 ~]$ curl -fsSL https://rpm.nodesource.com/setup_24.x | sudo bash -  
2026-05-26 08:45:21 - Cleaning up old repositories...  
2026-05-26 08:45:21 - Old repositories removed  
2026-05-26 08:45:21 - Supported architecture: x86_64  
2026-05-26 08:45:21 - Added NJSolid repository for LTS version: 24.x  
2026-05-26 08:45:21 - dnf available, updating...  
Node.js Packages for Linux RPM based distros - x86_64  
Metadata cache created.  
NJSolid Packages for Linux RPM based distros - x86_64  
Metadata cache created.  
2026-05-26 08:45:22 - Repository is configured and updated.  
2026-05-26 08:45:22 - You can use NJSolid Runtime as a node.js alternative  
2026-05-26 08:45:22 - To install NJSolid Runtime, run: dnf install nsolid -y  
2026-05-26 08:45:22 - Run: dnf install nodejs -y to complete the installation.  
[ec2-user@ip-10-0-30-78 ~]$ sudo dnf install nodejs -y  
Last metadata expiration check: 2026-05-26 08:45:22 on Tue May 26 08:45:22 2026.  
Dependencies resolved.  
===== Package Architecture Version Repository Size =====  
Installing:  
nodejs x86_64 2:24.16.0-1nodesource nodesource-nodejs 45 M  
  
Transaction Summary  
===== Package Architecture Version Repository Size =====  
Install 1 Package  
  
Total download size: 45 M  
Installed size: 130 M  
Downloading Packages:  
nodejs-24.16.0-1nodesource.x86_64.rpm 126 MB/s | 45 MB 00:00  
-----  
Total 125 MB/s | 45 MB 00:00  
Node.js Packages for Linux RPM based distros - x86_64 149 kB/s | 3.1 kB 00:00  
Importing GPG key 0x3AF28A14:  
  
Running scriptlet: nodejs-2:24.16.0-1nodesource.x86_64  
Verifying : nodejs-2:24.16.0-1nodesource.x86_64  
  
Installed:  
nodejs-2:24.16.0-1nodesource.x86_64  
  
Complete!  
[ec2-user@ip-10-0-30-78 ~]$ node -v  
v24.16.0  
[ec2-user@ip-10-0-30-78 ~]$
```

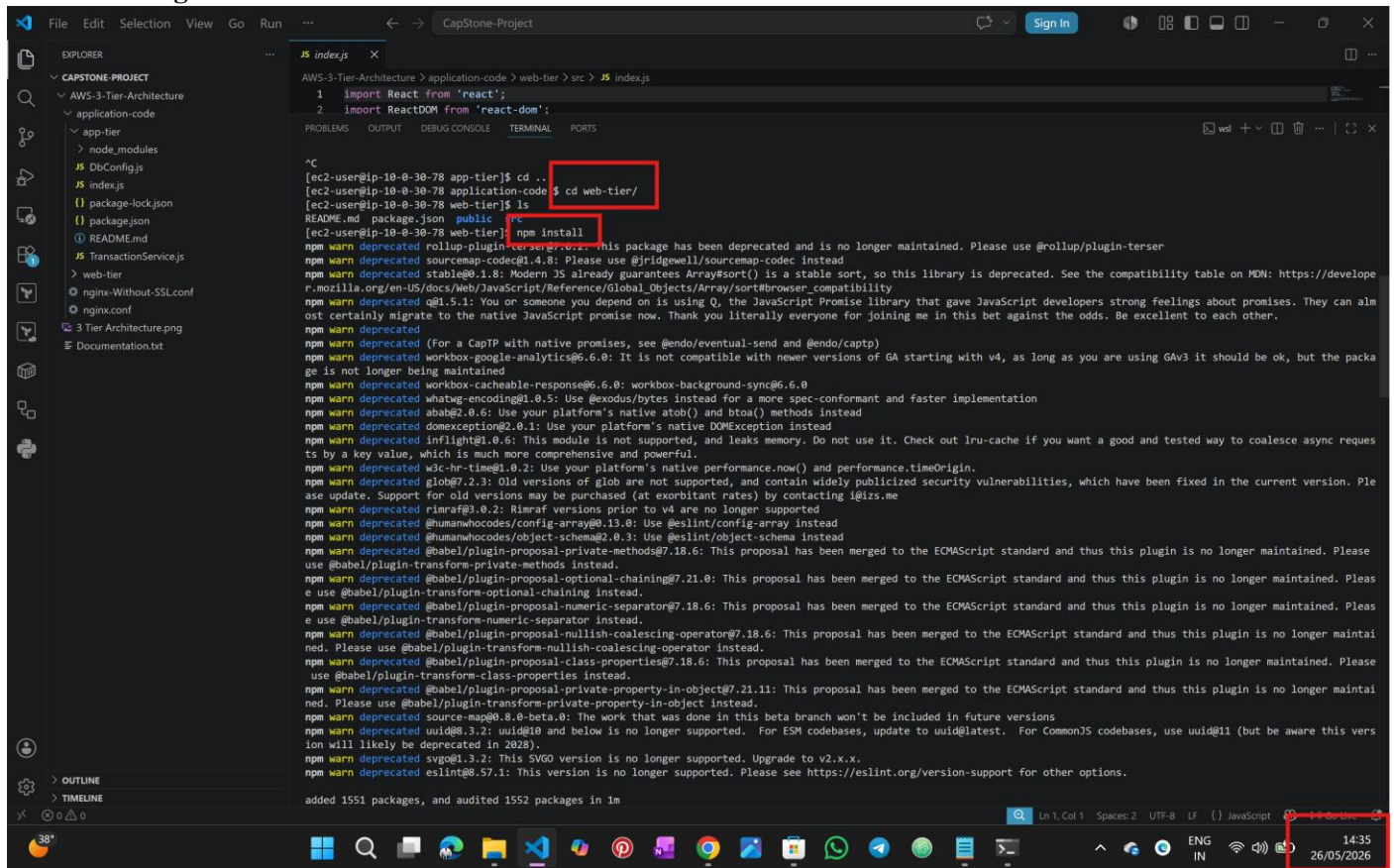
Cloned the git repo and deployed the backend I got some issues/ vulnerabilities I have fixed them using **npm audit fix --force**

```
[ec2-user@ip-10-0-30-78 ~]$ git clone https://github.com/rahnulathati-21/AWS-3-Tier-Architecture.git  
Cloning into 'AWS-3-Tier-Architecture'...  
remote: Enumerating objects: 89, done.  
remote: Counting objects: 100% (89/89), done.  
remote: Compressing objects: 100% (15/15), done.  
remote: Total 89 (delta 10), reused 4 (delta 4), pack-reused 70 (from 2)  
Receiving objects: 100% (89/89), 368.95 KiB | 28.38 MiB/s, done.  
Resolving deltas: 100% (13/13), done.  
[ec2-user@ip-10-0-30-78 ~]$ ls  
AWS-3-Tier-Architecture  
[ec2-user@ip-10-0-30-78 ~]$ cd AWS-3-Tier-Architecture/  
[ec2-user@ip-10-0-30-78 AWS-3-Tier-Architecture]$ ls  
'3 Tier Architecture.png' Documentation.txt application-code  
[ec2-user@ip-10-0-30-78 AWS-3-Tier-Architecture]$ cd application-code/  
[ec2-user@ip-10-0-30-78 application-code]$ ls  
app-tier nginx-Without-SSL.conf nginx.conf web-tier  
[ec2-user@ip-10-0-30-78 application-code]$ git branch  
* main  
[ec2-user@ip-10-0-30-78 application-code]$ git checkout -b rahnul-3-tire  
Switched to a new branch 'rahnul-3-tire'  
[ec2-user@ip-10-0-30-78 application-code]$ git branch  
main  
* rahnul-3-tire  
[ec2-user@ip-10-0-30-78 application-code]$ ls  
app-tier nginx-Without-SSL.conf nginx.conf web-tier  
[ec2-user@ip-10-0-30-78 application-code]$ cd app-tier/  
[ec2-user@ip-10-0-30-78 app-tier]$ npm install  
  
added 68 packages, and audited 69 packages in 1s  
  
2 packages are looking for funding  
run 'npm fund' for details  
  
7 vulnerabilities (3 low, 4 high)  
  
To address all issues, run:  
npm audit fix  
  
Run 'npm audit' for details.  
npm notice  
npm notice New minor version of npm available! 11.13.0 -> 11.15.0  
npm notice Changelog: https://github.com/npm/cli/releases/tag/v11.15.0  
npm notice To update run: npm install -g npm@11.15.0  
npm notice  
[ec2-user@ip-10-0-30-78 app-tier]$ node index.js  
AB3 backend app listening at http://localhost:4000
```

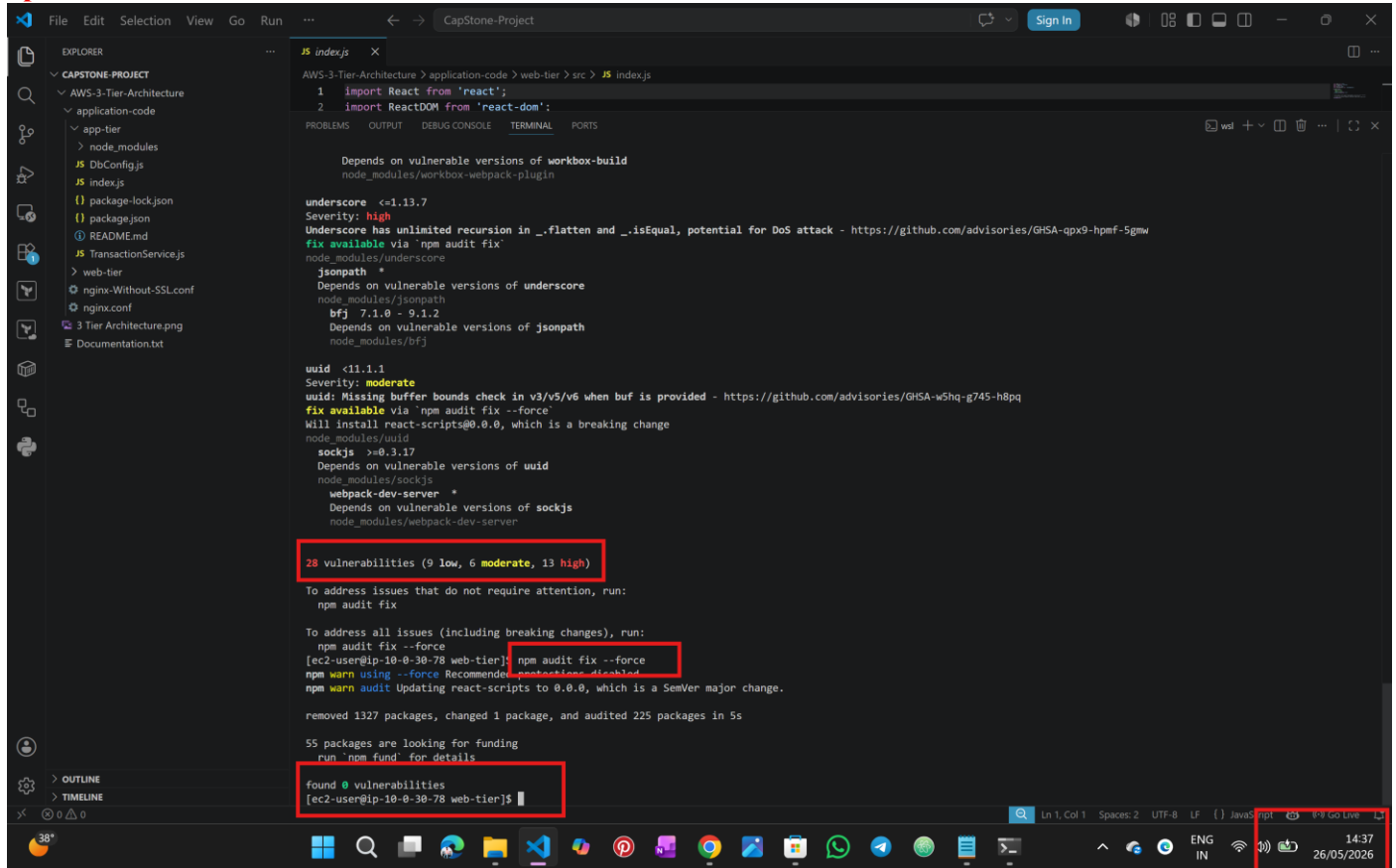
Back end URL is working as expected



frontend using ec2:



I got some issues/ vulnerabilities I have fixed them using
npm audit fix --force



```
File Edit Selection View Go Run ... CapStone-Project Sign In
EXPLORER
CAPSTONE-PROJECT
  application-code
    app-tier
      node_modules
        DbConfig.js
        index.js
        package-lock.json
        package.json
        README.md
        TransactionService.js
      web-tier
        nginx-Without-SSL.conf
        nginx.conf
        3 Tier Architecture.png
        Documentation.txt
  JS index.js
    1 import React from 'react';
    2 import ReactDOM from 'react-dom';

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Depends on vulnerable versions of workbox-build
node_modules/workbox-webpack-plugin

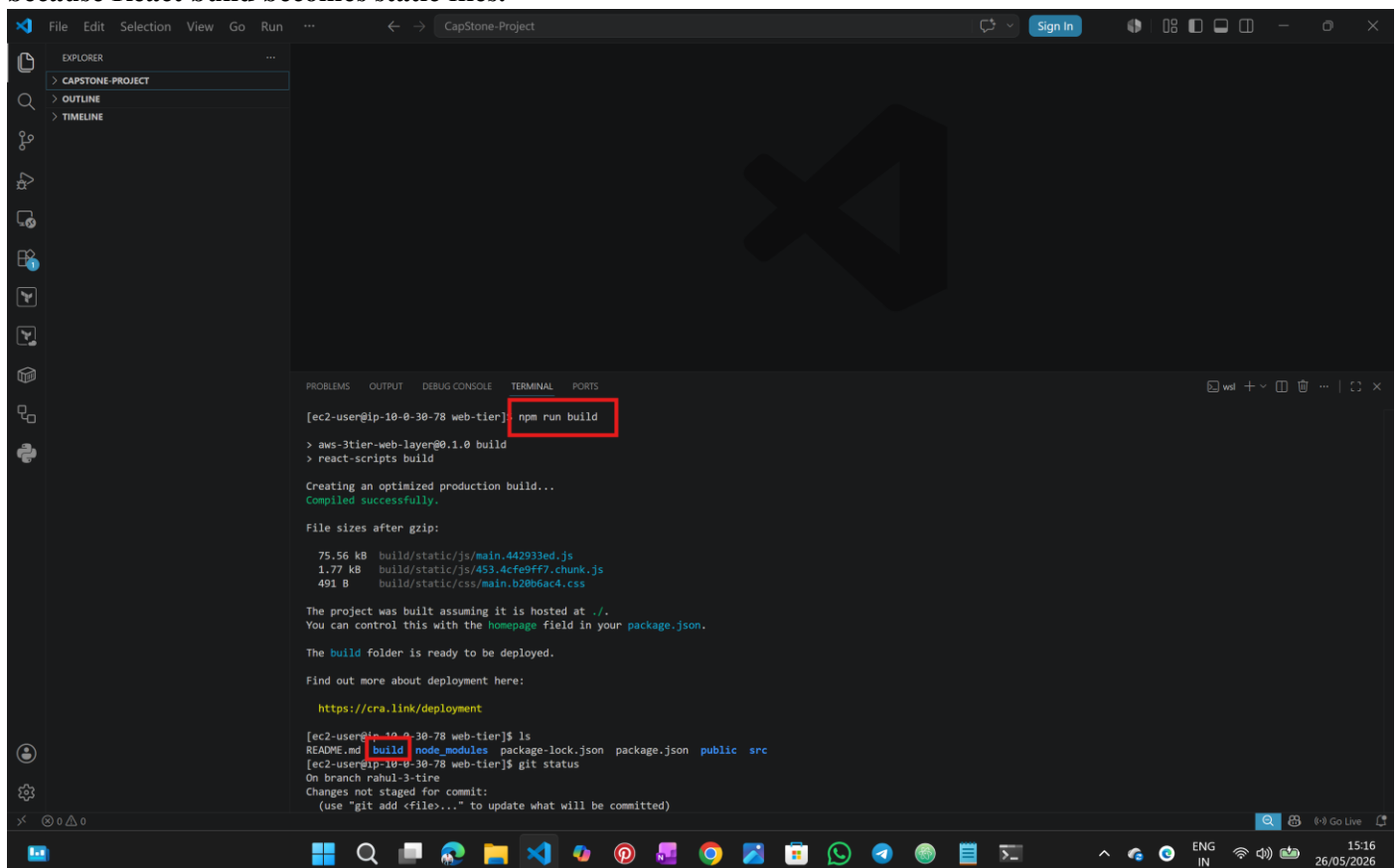
underscore <=1.13.7
Severity: high
Underscore has unlimited recursion in _.flatten and _.isEqual, potential for DoS attack - https://github.com/advisories/GHSA-gpx9-hpmf-5gmw
Fix available via 'npm audit fix'
node_modules/underscore
  jsonpath *
    Depends on vulnerable versions of underscore
    node_modules/jsonpath
      bfj 7.1.0 - 9.1.2
        Depends on vulnerable versions of jsonpath
        node_modules/bfj

uuid <11.1.1
Severity: moderate
uuid: Missing buffer bounds check in v3/v5/v6 when buf is provided - https://github.com/advisories/GHSA-wShq-g745-h8pq
Fix available via 'npm audit fix --force'
Will install react-scripts@0.0.0, which is a breaking change
node_modules/uuid
  sockjs >=0.3.17
    Depends on vulnerable versions of uuid
    node_modules/sockjs
      webpack-dev-server *
        Depends on vulnerable versions of sockjs
        node_modules/webpack-dev-server

28 vulnerabilities (9 low, 6 moderate, 13 high)
To address issues that do not require attention, run:
  npm audit fix

To address all issues (including breaking changes), run:
  npm audit fix --force
[ec2-user@ip-10-0-30-78 web-tier] npm audit fix --force
npm warn using --force Recommended protections disabled
npm warn audit Updating react-scripts to 0.0.0, which is a SemVer major change.
removed 1327 packages, changed 1 package, and audited 225 packages in 5s
55 packages are looking for funding
  run `npm fund` for details
found 0 vulnerabilities
[ec2-user@ip-10-0-30-78 web-tier]$
```

Since we are using the static web hosting we need to build and upload
because React build becomes static files.



```
File Edit Selection View Go Run ... CapStone-Project Sign In
EXPLORER
CAPSTONE-PROJECT
  OUTLINE
  TIMELINE

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
[ec2-user@ip-10-0-30-78 web-tier] npm run build
> aws-3tier-web-layer@0.1.0 build
> react-scripts build

Creating an optimized production build...
Compiled successfully.

File sizes after gzip:
  75.56 kB build/static/js/main.442933ed.js
  1.77 kB build/static/js/453.4cfe9ff7.chunk.js
  491 B build/static/css/main.b20b6ac4.css

The project was built assuming it is hosted at ./
You can control this with the homepage field in your package.json.

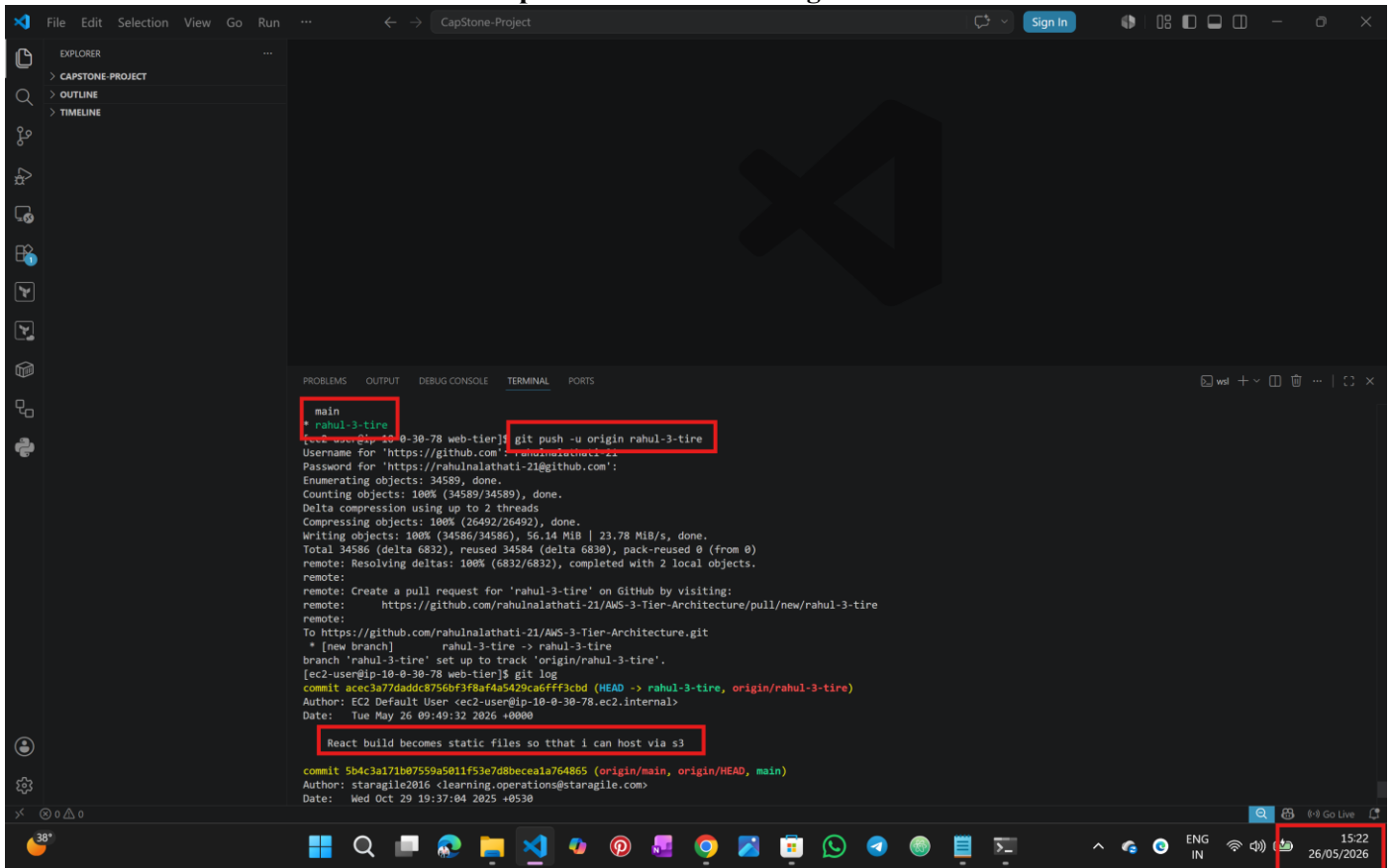
The build folder is ready to be deployed.

Find out more about deployment here:

  https://cra.link/deployment

[ec2-user@ip-10-0-30-78 web-tier]$ ls
README.md build node_modules package-lock.json package.json public src
[ec2-user@ip-10-0-30-78 web-tier]$ git status
On branch rahul-3-tire
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
```

To Download the build files to local and upload the s3 I have use github

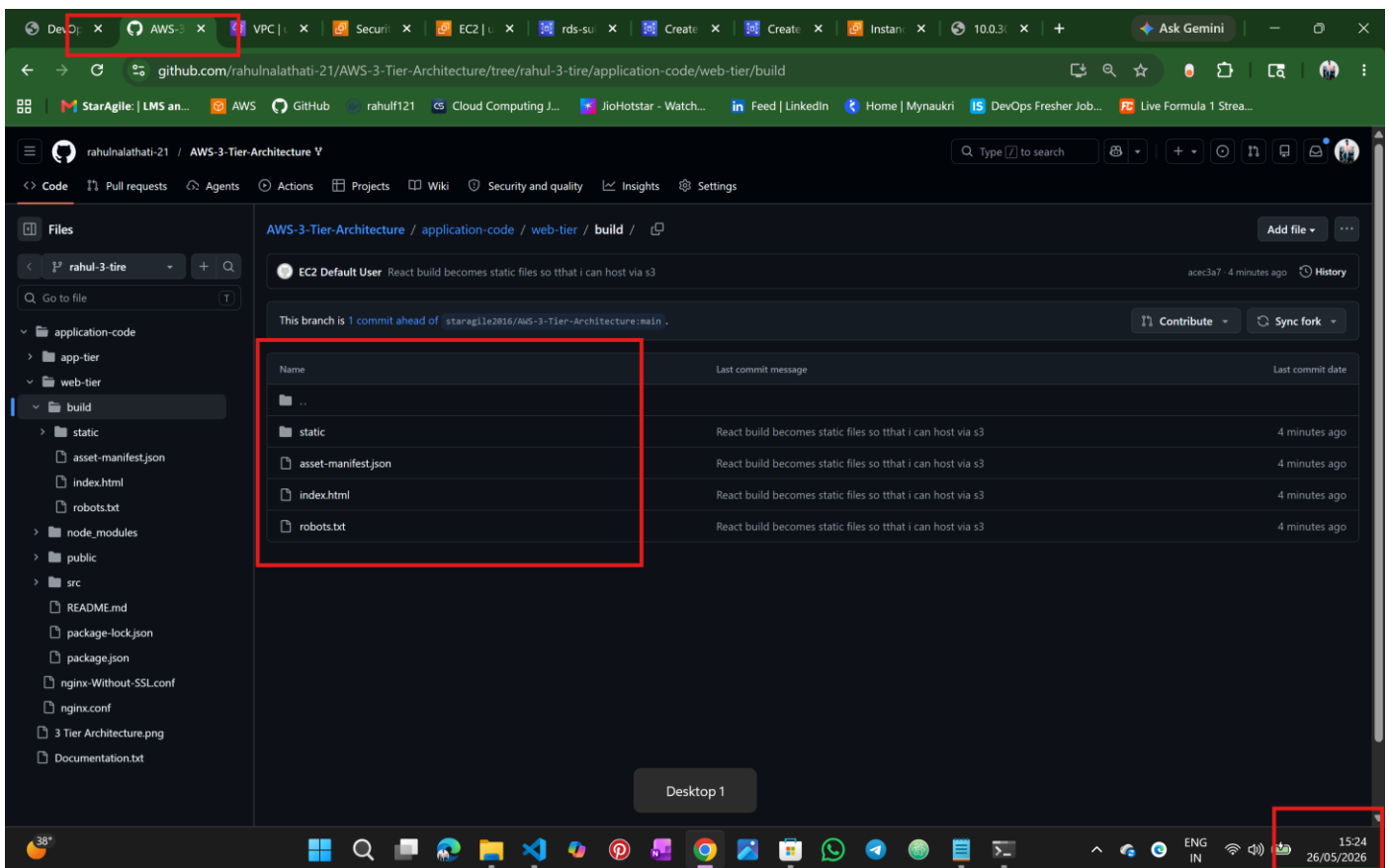


The screenshot shows the VS Code interface with a terminal window open. The terminal displays the output of a `git push` command. The commit message is "React build becomes static files so tthat i can host via s3". The commit hash is `5b4c3a171b07559a5011f53e7d8bece1a764865`. The commit is pushed to the `main` branch of the repository `https://github.com/rahnalathati-21/AWS-3-Tier-Architecture`. The commit is created as a pull request on GitHub.

```
main
* rahul-3-tire
[ec2-user@ip-10-0-30-78 web-tier]$ git push -u origin rahul-3-tire
Username for 'https://github.com': rahulathati-21
Password for 'https://github.com':
Enumerating objects: 34589, done.
Counting objects: 100% (34589/34589), done.
Delta compression using up to 2 threads
Compressing objects: 100% (26492/26492), done.
Writing objects: 100% (34586/34586), 56.14 MiB | 23.78 MiB/s, done.
Total 34586 (delta 6832), reused 34584 (delta 6830), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (6832/6832), completed with 2 local objects.
remote:
remote: Create a pull request for 'rahul-3-tire' on GitHub by visiting:
remote:   https://github.com/rahnalathati-21/AWS-3-Tier-Architecture/pull/new/rahul-3-tire
remote:
To https://github.com/rahnalathati-21/AWS-3-Tier-Architecture.git
 * [new branch]      rahul-3-tire -> rahul-3-tire
branch 'rahul-3-tire' set up to track 'origin/rahul-3-tire'.
[ec2-user@ip-10-0-30-78 web-tier]$ git log
commit acec3a77daddc8756bf3f8af4a5429ca6fff3cbd (HEAD -> rahul-3-tire, origin/rahul-3-tire)
Author: EC2 Default User <ec2-user@ip-10-0-30-78.ec2.internal>
Date:   Tue May 26 09:49:32 2026 +0000

    React build becomes static files so tthat i can host via s3

commit 5b4c3a171b07559a5011f53e7d8bece1a764865 (origin/main, origin/HEAD, main)
Author: staragile2016 <learning.operations@staragile.com>
Date:   Wed Oct 29 19:37:04 2025 +0530
```



The screenshot shows the GitHub repository page for `rahnalathati-21 / AWS-3-Tier-Architecture`. The `build` directory is selected, showing the commit `5b4c3a171b07559a5011f53e7d8bece1a764865` with the message "React build becomes static files so tthat i can host via s3". The commit is 4 minutes old. The commit message is "React build becomes static files so tthat i can host via s3". The commit is pushed to the `main` branch of the repository `https://github.com/rahnalathati-21/AWS-3-Tier-Architecture`. The commit is created as a pull request on GitHub.

Name	Last commit message	Last commit date
..		
static	React build becomes static files so tthat i can host via s3	4 minutes ago
asset-manifest.json	React build becomes static files so tthat i can host via s3	4 minutes ago
index.html	React build becomes static files so tthat i can host via s3	4 minutes ago
robots.txt	React build becomes static files so tthat i can host via s3	4 minutes ago

us-east-1.console.aws.amazon.com/s3/upload/rahul-3tier-frontend?region=us-east-1

StarAgile: | LMS an... | AWS | GitHub | rahul121 | Cloud Computing J... | JioHotstar - Watch... | Feed | LinkedIn | Home | Mynaukri | DevOps Fresher Job... | Live Formula 1 Stre...

Amazon S3

Upload succeeded
For more information, see the Files and folders table.

Upload: status
After you navigate away from this page, the following information is no longer available.

Summary

Destination
s3://rahul-3tier-frontend

Succeeded
11 Files, 1.1 MB (100.00%)

Failed
0 Files, 0 B (0%)

Files and folders

Configuration

Files and folders (11 total, 1.1 MB)

Find by name

Name	Folder	Type	Size	Status	Error
3TierArch.bcbbf1a8db1497bf459...	static/media/	image/png	124.7 KB	Succeeded	-
453.4cfe9f7.chunk.js	static/js/	text/javascript	4.4 KB	Succeeded	-
453.4cfe9f7.chunk.js.map	static/js/	-	10.3 KB	Succeeded	-
main.442933ed.js	static/js/	text/javascript	224.8 KB	Succeeded	-
main.442933ed.js.LICENSE.txt	static/js/	text/plain	1.4 KB	Succeeded	-
main.442933ed.js.map	static/js/	-	767.5 KB	Succeeded	-
main.b20b6ac4.css	static/css/	text/css	829.0 B	Succeeded	-
main.b20b6ac4.css.map	static/css/	-	1.5 KB	Succeeded	-
asset-manifest.json	-	application/json	611.0 B	Succeeded	-
index.html	-	text/html	514.0 B	Succeeded	-

CloudShell | Feedback | Console Mobile App | © 2026, Amazon Web Services, Inc. or its affiliates. | Privacy | Terms | Cookie preferences | 15:36 | 26/05/2026

us-east-1.console.aws.amazon.com/s3/bucket/rahul-3tier-frontend/property/website/edit?region=us-east-1

StarAgile: | LMS an... | AWS | GitHub | rahul121 | Cloud Computing J... | JioHotstar - Watch... | Feed | LinkedIn | Home | Mynaukri | DevOps Fresher Job... | Live Formula 1 Stre...

Amazon S3 > Buckets > rahul-3tier-frontend > Edit static website hosting

Static website hosting
Use this bucket to host a website or redirect requests. [Learn more](#)

Static website hosting
☐ Disable
☒ Enable

Hosting type
☒ Host a static website
Use the bucket endpoint as the web address. [Learn more](#)
☐ Redirect requests for an object
Redirect requests to another bucket or domain. [Learn more](#)

For your customers to access content at the website endpoint, you must make all your content publicly readable. To do so, you can edit the S3 Block Public Access settings for the bucket. For more information, see [Using Amazon S3 Block Public Access](#)

Index document
Specify the home or default page of the website.
index.html

Error document - optional
This is returned when an error occurs.
error.html

Redirection rules - optional
Redirection rules, written in S3DN, automatically redirect webpage requests for specific content. [Learn more](#)

CloudShell | Feedback | Console Mobile App | © 2026, Amazon Web Services, Inc. or its affiliates. | Privacy | Terms | Cookie preferences | 15:37 | 26/05/2026

us-east-1.console.aws.amazon.com/s3/bucket/rahul-3tier-frontend/property/policy/edit?region=us-east-1

Edit bucket policy

The bucket policy, written in JSON, provides access to the objects stored in the bucket. Bucket policies don't apply to objects owned by other accounts. [Learn more](#)

Bucket ARN
arn:aws:s3::rahul-3tier-frontend

Policy

```
1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Sid": "PublicReadGetObject",
6       "Effect": "Allow",
7       "Principal": "*",
8       "Action": "s3:GetObject",
9       "Resource": "arn:aws:s3::rahul-3tier-frontend/*"
10    }
11  ]
12 }
```

Edit statement

Select a statement
Select an existing statement in the policy or add a new statement.

[+ Add new statement](#)

38°

15:39
26/05/2026

us-east-1.console.aws.amazon.com/s3/buckets/rahul-3tier-frontend?region=us-east-1&tab=properties

Transfer acceleration
Use an accelerated endpoint for faster data transfers. [Learn more](#)
Transfer acceleration
Disabled

Object Lock
Store objects using a write-once-read-many (WORM) model to help you prevent objects from being deleted or overwritten for a fixed amount of time or indefinitely. Object Lock works only in versioned buckets. [Learn more](#)
Object Lock
Disabled

Requester pays
When enabled, the requester pays for requests and data transfer costs, and anonymous access to this bucket is disabled. [Learn more](#)
Requester pays
Disabled

Static website hosting
Use this bucket to host a website or redirect requests. [Learn more](#)
[We recommend using AWS Amplify Hosting for static website hosting](#)
Deploy a fast, secure, and reliable website quickly with AWS Amplify Hosting. [Learn more about Amplify Hosting](#) or [View your existing Amplify apps](#). [Create Amplify app](#)

S3 static website hosting
Enabled

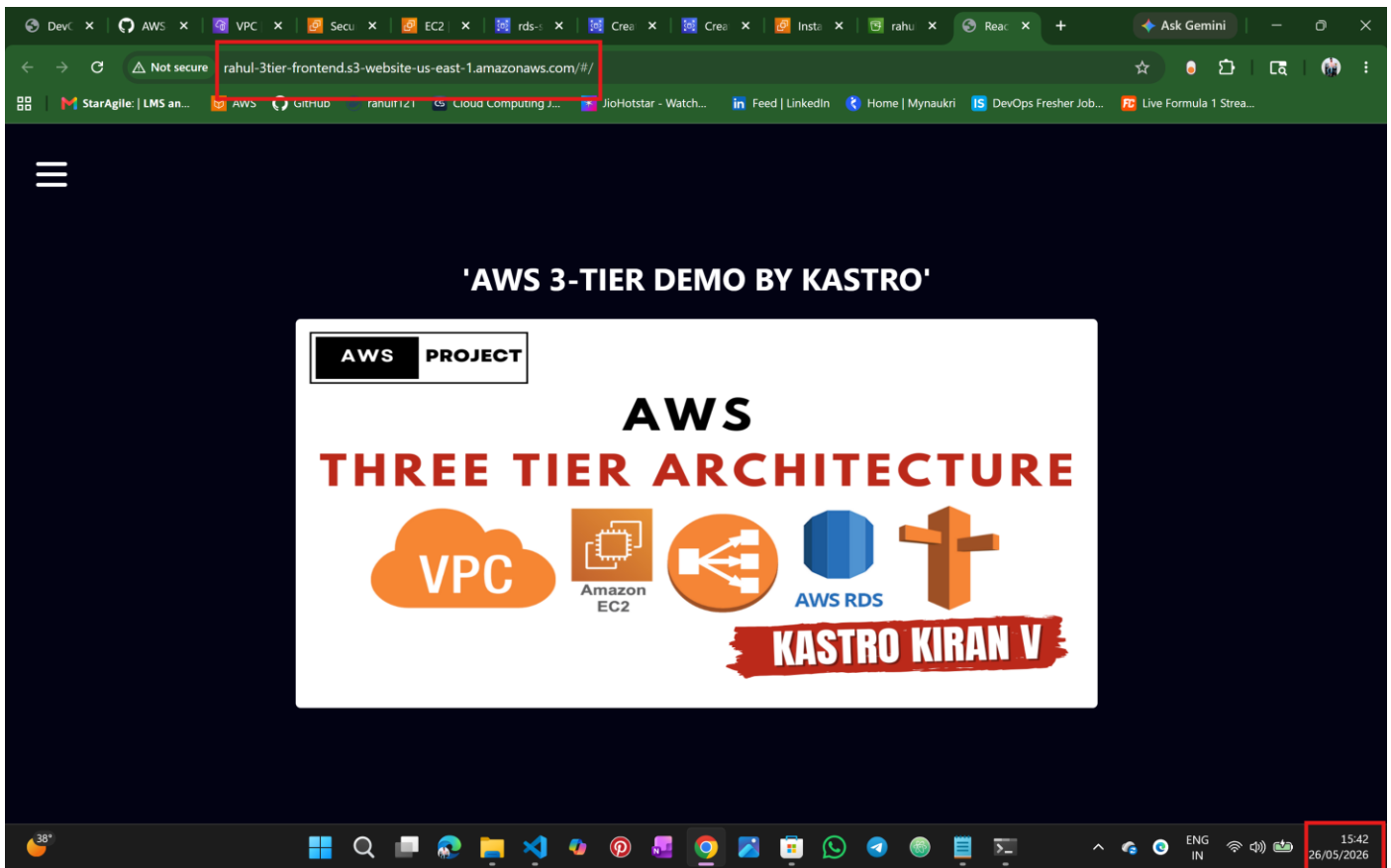
Hosting type
Bucket hosting

Bucket website endpoint
When you configure your bucket as a static website, the website is available at the AWS Region-specific website endpoint of the bucket. [Learn more](#)
<http://rahul-3tier-frontend.s3-website-us-east-1.amazonaws.com>

CloudShell Feedback Console Mobile App

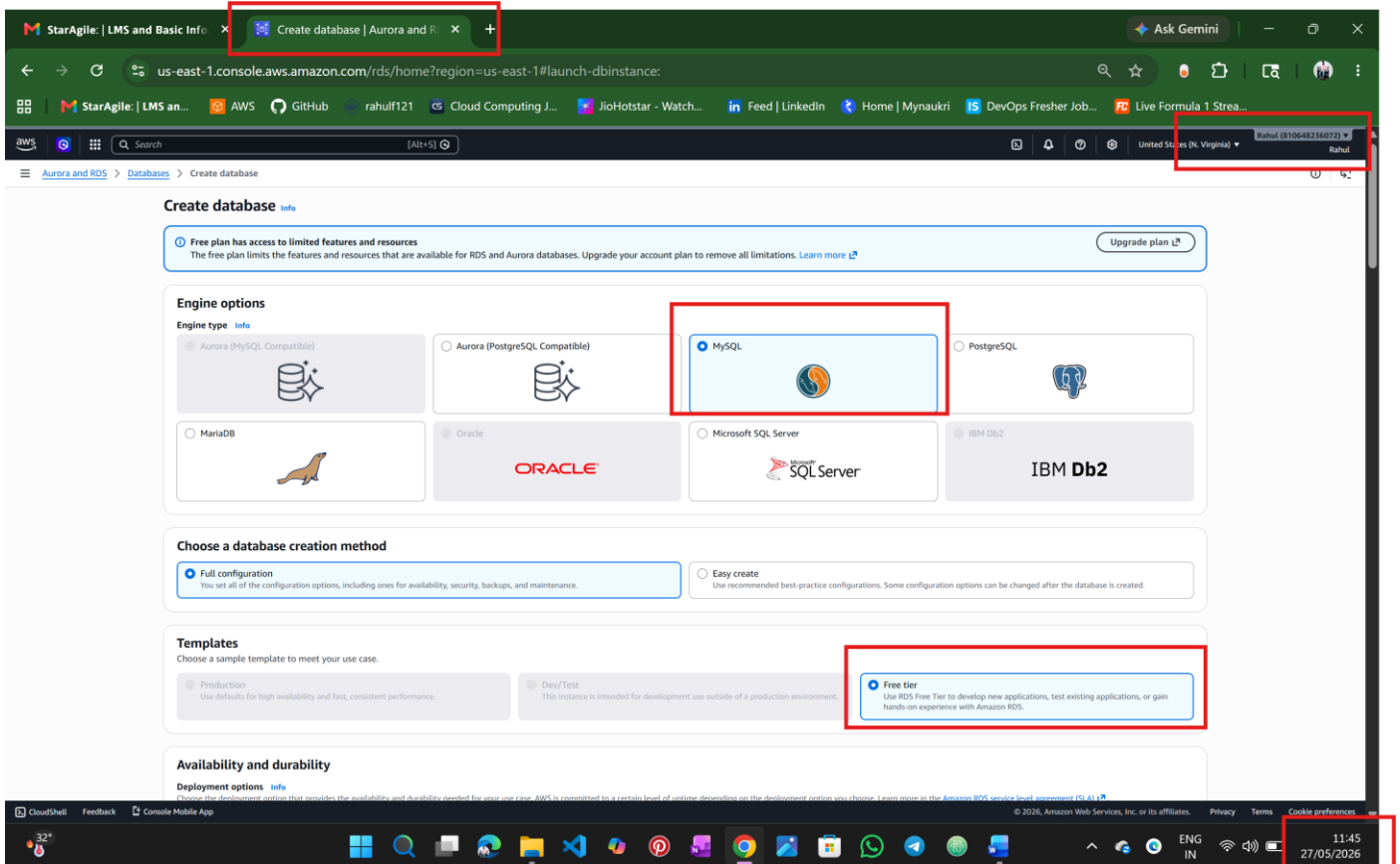
© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

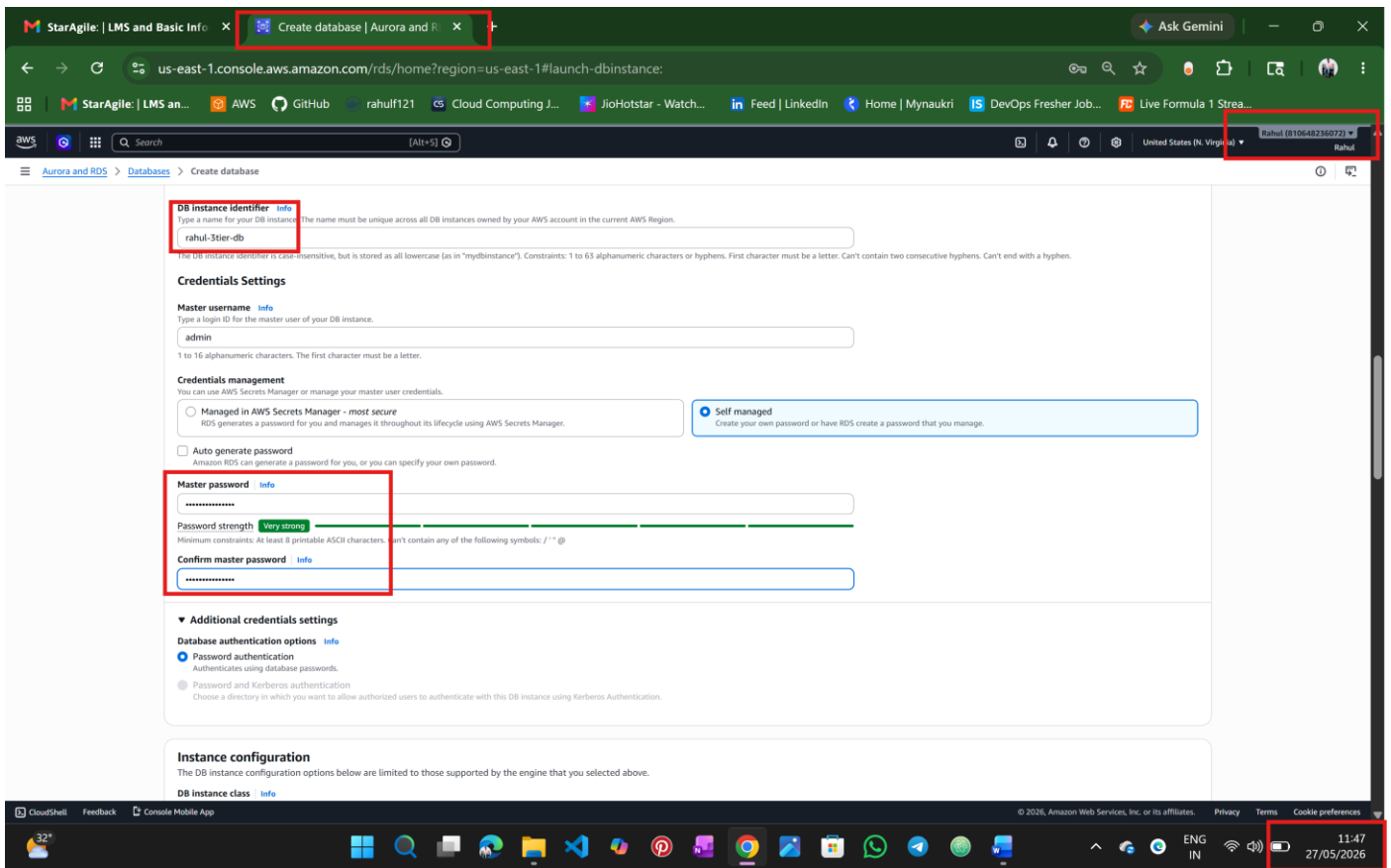
15:41
26/05/2026



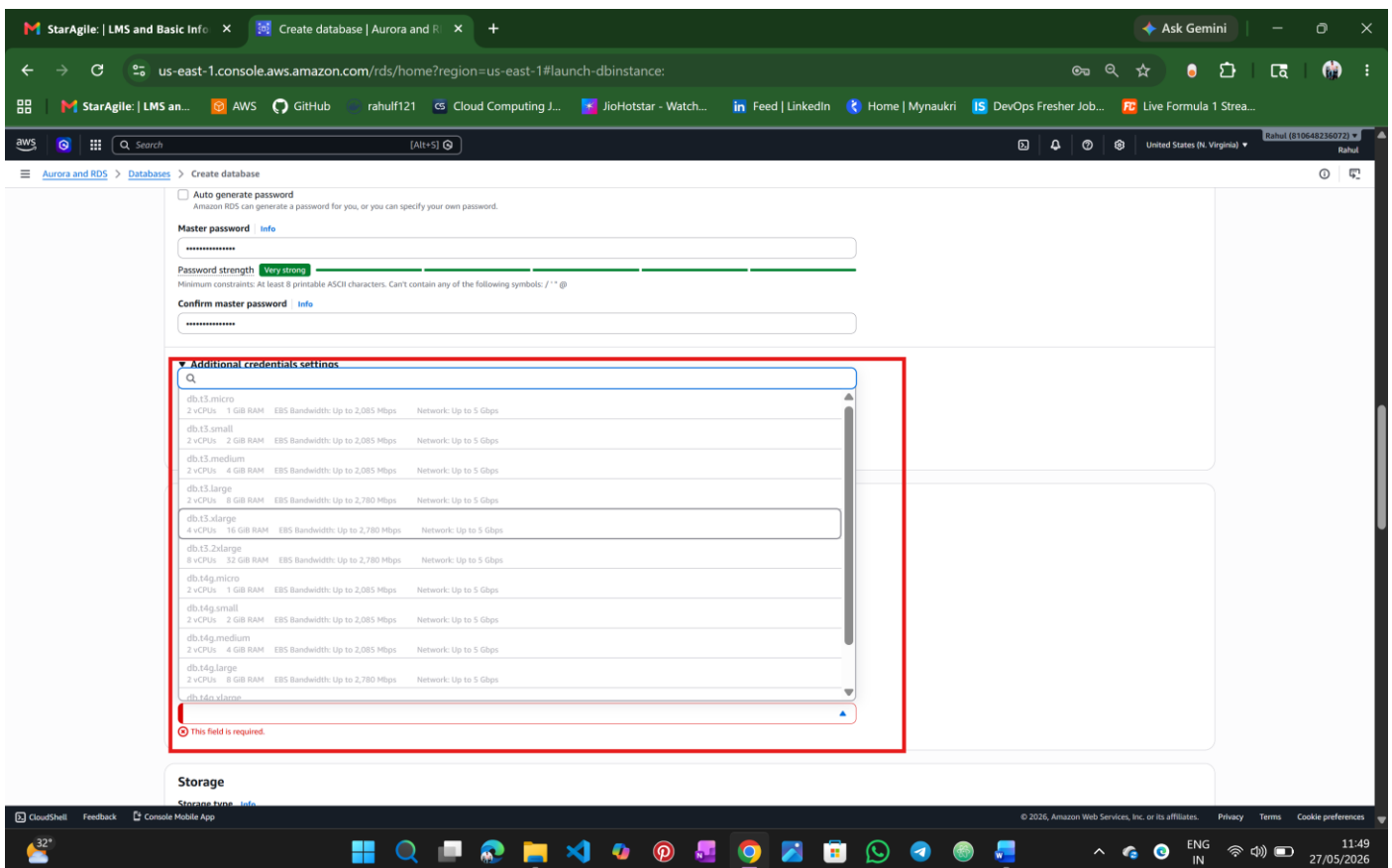
Step 4: Create RDS MySQL Database

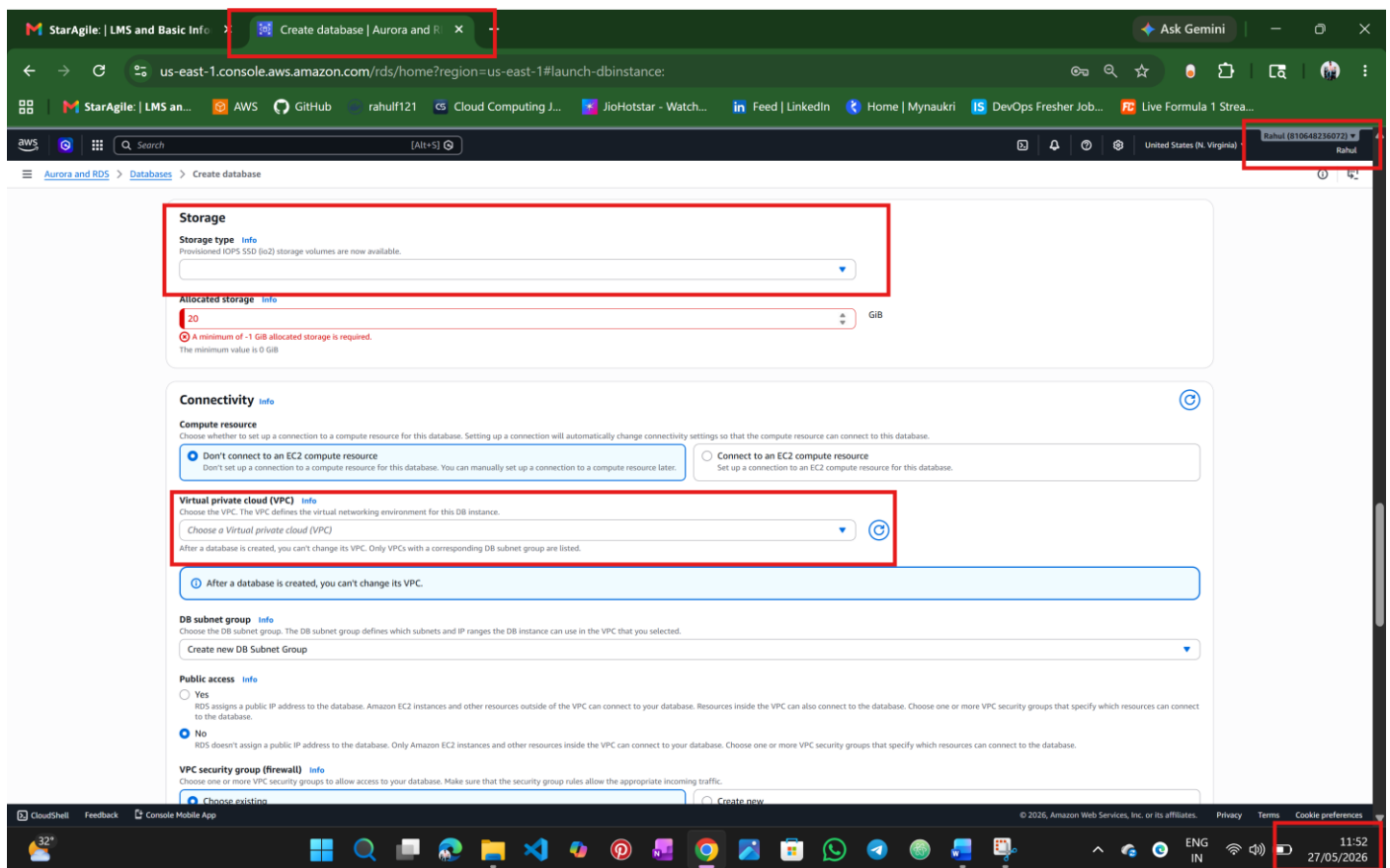
AWS Console → RDS → Create Database





Note: to select the Instance type and VPC (can't able to select the default also), so without implementation will move theoretically





Configuration

Setting	Value
Database Engine	MySQL
Template	Free Tier
Deployment	Single-AZ
DB Name	transactionsdb
Username	admin
Public Access	No
Security Group	database-sg

Database deployed in Private Subnet only

Step 5: Create Database Table

Connect from EC2:

`mysql -h RDS_ENDPOINT -u admin -p` (after RDS create it will show this end point)

it will ask for which DB

USE transactionsdb;

Create table:

```
CREATE TABLE transactions (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  amount INT,  
  description VARCHAR(255)  
);
```

Verify:

```
SHOW TABLES;
```

Step 6: EC2 Backend Server

Configure Database Connection(we use VIM command to edit the DB credentials)

Vim TransactionService.js

```
host: "RDS_ENDPOINT",  
user: "admin",  
password: "PASSWORD",  
database: "transactionsdb"
```

Start Backend Server

```
node index.js
```

Expected:

AB3 backend app listening at http://localhost:4000

Test Backend

http:// 3.80.148.145:4000/health

Expected:

"This is the health check"

Step 7: Connect Frontend to Backend

```
src/components/DatabaseDemo/DatabaseDemo.js
```

Replace:

```
'/api/transaction'
```

With:

'http:// 3.80.148.145:4000/transaction'

Rebuild Frontend

`npm run build`

Reupload Frontend Build to S3

Upload updated build contents again.

Final Testing

Open S3 website endpoint.

Test:

- Add transaction
- View transactions
- Delete transactions
- Refresh page

Verify:

- Data stored in RDS
- Frontend communicates with backend
- Backend communicates with database

Conclusion

This project successfully demonstrates a traditional AWS 3-tier architecture deployment using native AWS services.

The application is fully functional with:

- Frontend hosted on S3
- Backend running on EC2
- Database hosted on RDS
- Secure networking using VPC and Security Groups
- End-to-end communication between all tiers